Manual de Uso de Código

WEARO

Documentación técnica del código fuente

Total: 17.984 líneas de código | 88 archivos fuente

Versión 1.0 - Marzo 2026

Contenido

1. Resumen del proyecto 2
2. Estructura de directorios 3
3. Conteo de líneas por archivo 4
3.1. Backend: Configuración 4
3.2. Backend: Controladores 4
3.3. Backend: Middleware 5
3.4. Backend: Modelos 5
3.5. Backend: Rutas 6
3.6. Backend: Servicios 6
3.7. Frontend: Pantallas 7
3.8. Frontend: Componentes reutilizables 8
3.9. Frontend: Servicios API 8
3.10. Frontend: Store (Redux) 9
3.11. Frontend: Navegación y utilidades 9
3.12. Archivos de configuración 9
4. Explicación detallada de archivos 10
4.1. Backend 10
index.js (70 líneas) 10
config/db.js (40 líneas) 10
config/setup.sql (173 líneas) 10
Controladores (11 archivos, 1.721 líneas) 10
Middleware (3 archivos, 89 líneas) 12
Modelos (11 archivos, 1.776 líneas) 12
Servicios (2 archivos, 439 líneas) 14
4.2. Frontend 14
App.js (15 líneas) 14
Pantallas (17 archivos, 9.743 líneas) 14
Componentes reutilizables (6 archivos, 1.015 líneas) 17
Servicios API (11 archivos, 761 líneas) 18
Store Redux (10 archivos, 1.239 líneas) 19
Navegación: AppNavigator.js (194 líneas) 20
Hook: useTheme.js (140 líneas) 20
5. Resumen de totales 20

1. Resumen del proyecto

Wearo es una aplicación móvil de gestión de armario personal y red social de moda. El proyecto consta de dos módulos principales: un backend (API REST) y un frontend (aplicación móvil), con un total de 17.984 líneas de código fuente distribuidas en 88 archivos.

El backend está construido con Node.js y Express, utiliza PostgreSQL como base de datos y se integra con servicios externos como Google Gemini (IA), OpenWeatherMap (clima) y Groq Whisper (transcripción de audio). El frontend es una aplicación React Native con Expo, que utiliza Redux Toolkit para la gestión de estado.

Distribución del código:

- Backend: 4.840 líneas en 41 archivos (27% del total).
- Frontend: 13.144 líneas en 47 archivos (73% del total).

2. Estructura de directorios

```
Wearo/
|-- backend/
|   |-- package.json                  # Dependencias del servidor
|   +-- src/
|       |-- index.js                   # Entry point del servidor
|       |-- config/
|       |   |-- db.js                  # Conexión a PostgreSQL
|       |   +-- setup.sql              # Esquema de la base de datos
|       |-- controllers/               # Lógica de negocio (11 archivos)
|       |-- middleware/                 # Auth, admin, uploads(3 archivos)
|       |-- models/                    # Consultas SQL (11 archivos)
|       |-- routes/                    # Definición de rutas (11 archivos)
|       +-- services/                  # IA y clima (2 archivos)
+-- frontend/
    |-- App.js                         # Entry point React Native
    |-- app.json                       # Configuración Expo
    |-- package.json                   # Dependencias del cliente
    +-- src/
        |-- components/                # Componentes reutilizables (6
archivos)
        |-- hooks/                     # Custom hooks (1 archivo)
        |-- navigation/                # Navegación (1 archivo)
        |-- screens/                   # 17 pantallas
        |-- services/                  # Llamadas a la API (11 archivos)
        +-- store/                     # Redux slices (10 archivos)
```

3. Conteo de líneas por archivo

3.1. Backend: Configuración

Archivo	Líneas	Descripción
src/index.js	70	Entry point, Express server, rutas y middleware
src/config/db.js	40	Pool de conexiones PostgreSQL con SSL
src/config/setup.sql	173	Esquema completo: 14 tablas SQL
SUBTOTAL	**283**	

3.2. Backend: Controladores

Archivo	Líneas	Descripción
controllers/admin.controller.js	189	Gestión usuarios, crear admins, inhabilitar
controllers/ai.controller.js	103	Recomendación IA, chat, sugerencias compra
controllers/auth.controller.js	205	Registro, login, refresh token, logout
controllers/calendar.controller.js	145	CRUD calendario, planificación outfits
controllers/garments.controller.js	176	CRUD prendas, favoritos, upload imagen
controllers/messages.controller.js	169	Conversaciones, mensajes, audio, fotos
controllers/outfits.controller.js	135	CRUD outfits, asociar prendas
controllers/profile.controller.js	262	Perfil, follows, seguidores, avatar
controllers/social.controller.js	176	Feed, likes, comentarios, publicar
controllers/stats.controller.js	111	Estadísticas de armario y uso
controllers/support.controller.js	50	Tickets de soporte de usuarios
SUBTOTAL	**1721**	

3.3. Backend: Middleware

Archivo	Líneas	Descripción
middleware/auth.middleware.js	32	Verificación de JWT en peticiones
middleware/admin.middleware.js	18	Verificación de rol administrador
middleware/upload.middleware.js	39	Configuración Multer para subida de archivos
SUBTOTAL	**89**	

3.4. Backend: Modelos

Archivo	Líneas	Descripción
models/admin.model.js	241	Queries admin: usuarios, stats, acciones
models/calendar.model.js	112	Queries calendario: entries, outfits día
models/follow.model.js	233	Queries follows, solicitudes, admins compartidos
models/garment.model.js	98	Queries CRUD prendas, colores, favoritos
models/messages.model.js	189	Queries conversaciones, mensajes, no leídos
models/outfit.model.js	151	Queries CRUD outfits, prendas asociadas
models/social.model.js	349	Queries feed, likes, comentarios, respuestas
models/stats.model.js	99	Queries estadísticas armario
models/support.model.js	77	Queries tickets soporte
models/user-profile.model.js	178	Queries perfil, búsqueda, onboarding
models/user.model.js	49	Queries registro, login, buscar por email
SUBTOTAL	**1776**	

3.5. Backend: Rutas

Archivo	Líneas	Descripción
routes/admin.routes.js	32	Rutas panel administración
routes/ai.routes.js	20	Rutas IA: recomendar, chat, compras
routes/auth.routes.js	18	Rutas auth: login, register, refresh, logout
routes/calendar.routes.js	27	Rutas calendario: obtener, asignar, eliminar
routes/garments.routes.js	31	Rutas prendas: CRUD, favoritos
routes/messages.routes.js	32	Rutas mensajes: conversaciones, enviar, audio
routes/outfits.routes.js	41	Rutas outfits: CRUD, compartir
routes/profile.routes.js	29	Rutas perfil: ver, editar, follows
routes/social.routes.js	49	Rutas sociales: feed, likes, comentarios
routes/stats.routes.js	26	Rutas estadísticas
routes/support.routes.js	7	Rutas soporte: crear ticket
SUBTOTAL	**312**	

3.6. Backend: Servicios

Archivo	Líneas	Descripción
services/ai.service.js	329	Integración Google Gemini: prompts, contexto armario
services/weather.service.js	110	Integración OpenWeatherMap: clima, ciudades
SUBTOTAL	**439**	

3.7. Frontend: Pantallas

Archivo	Líneas	Descripción
screens/SocialScreen.js	1563	Feed social: descubrir, siguiendo, likes, comentarios
screens/ProfileScreen.js	1392	Perfil usuario: posts, seguidores, solicitudes
screens/AdminScreen.js	1112	Panel admin: usuarios, contenido, soporte, stats
screens/ChatScreen.js	992	Chat DM: texto, fotos, audio, posts compartidos
screens/CalendarScreen.js	545	Calendario: planificación mensual outfits
screens/HomeScreen.js	509	Inicio: clima, IA, stats rápidas, outfit del día
screens/AddGarmentScreen.js	488	Formulario alto/edición prenda con color picker
screens/OnboardingScreen.js	419	Config inicial: username, genero, estilos
screens/OutfitDetailScreen.js	412	Detalle outfit: ver, editar, eliminar
screens/OutfitsScreen.js	331	Lista outfits: filtros, IA chat, "que me falta"
screens/WardrobeScreen.js	315	Armario: grid prendas, filtros categoría/temporada
screens/StatsScreen.js	288	Estadísticas: gráficos, distribución, uso
screens/RegisterScreen.js	280	Registro: validación, creación cuenta
screens/MessagesScreen.js	266	Lista conversaciones: búsqueda, no leídos
screens/CreateOutfitScreen.js	256	Crear outfit: selección prendas, portada
screens/LoginScreen.js	247	Login: email, contraseña, errores
screens/EditProfileScreen.js	181	Editar perfil: avatar, nombre, bio
screens/GarmentDetailScreen.js	147	Detalle prenda: info, editar, eliminar
SUBTOTAL	**9743**	

3.8. Frontend: Componentes reutilizables

Archivo	Líneas	Descripción
components/AIChatModal.js	602	Modal chat IA: mensajes, grabación, sugerencias
components/GarmentCarousel.js	174	Carrusel horizontal imágenes prendas
components/ScreenHeader.js	86	Cabecera reutilizable con título y acciones
components/EmptyState.js	63	Estado vacío con icono, texto y botón
components/StatCard.js	58	Tarjeta estadística con icono y valor
components/Card.js	32	Componente tarjeta contenedora genérica
SUBTOTAL	**1015**	

3.9. Frontend: Servicios API

Archivo	Líneas	Descripción
services/api.js	70	Axios instance, interceptor refresh token, baseURL
services/garments.service.js	98	Llamadas API prendas: CRUD, favoritos, upload
services/outfits.service.js	88	Llamadas API outfits: CRUD, compartir
services/calendar.service.js	81	Llamadas API calendario: obtener, asignar
services/admin.service.js	81	Llamadas API admin: usuarios, stats, acciones
services/social.service.js	78	Llamadas API feed: posts, likes, comentarios
services/messages.service.js	74	Llamadas API mensajes: enviar, audio, fotos
services/profile.service.js	62	Llamadas API perfil: ver, editar, follows
services/auth.service.js	55	Llamadas API auth: login, register, refresh
services/ai.service.js	38	Llamadas API IA: recomendar, chat
services/stats.service.js	36	Llamadas API estadísticas
SUBTOTAL	**761**	

3.10. Frontend: Store (Redux)

Archivo	Líneas	Descripción
store/profileSlice.js	227	Estado perfil: datos, seguidores, solicitudes
store/messagesSlice.js	226	Estado mensajes: conversaciones, envió, polling
store/socialSlice.js	174	Estado social: feed, likes, comentarios
store/aiSlice.js	127	Estado IA: recomendación, chat, sugerencias
store/wardrobeSlice.js	116	Estado armario: prendas, filtros, favoritos
store/authSlice.js	114	Estado auth: usuario, token, login/logout
store/outfitsSlice.js	95	Estado outfits: lista, crear, editar
store/calendarSlice.js	82	Estado calendario: entries por día
store/statsSlice.js	53	Estado estadísticas: datos armario
store/index.js	25	Configuración del store Redux global
SUBTOTAL	**1239**	

3.11. Frontend: Navegación y utilidades

Archivo	Líneas	Descripción
navigation/AppNavigator.js	194	Stack + Tab navigator, rutas protegidas
hooks/useTheme.js	140	Hook tema claro/oscuro con AsyncStorage
App.js	15	Entry point: Provider Redux + ThemeContext
SUBTOTAL	**349**	

3.12. Archivos de configuración

Archivo	Líneas	Descripción
frontend/app.json	41	Config Expo: nombre, versión, apiUrl Railway
frontend/package.json	44	Dependencias frontend: React Native, Redux, Expo
backend/package.json	36	Dependencias backend: Express, pg, JWT, Multer
SUBTOTAL	**121**	

4. Explicación detallada de archivos

4.1. Backend

index.js (70 líneas)
Punto de entrada del servidor. Configura Express con los middlewares necesarios: CORS para permitir peticiones del frontend, express.json() para parsear el body de las peticiones, y express.static() para servir los archivos subidos (imágenes de prendas, avatares, audio). Registra todas las rutas bajo el prefijo /api/v1/ y arranca el servidor en el puerto definido por la variable de entorno PORT (por defecto 3000). También sirve los directorios de uploads como ficheros estáticos accesibles vía URL.

config/db.js (40 líneas)
Configura el pool de conexiones a PostgreSQL utilizando el paquete pg. Soporta dos modos de conexión: mediante la variable DATABASE_URL (cadena completa con SSL, usada en producción con Railway) o mediante variables individuales (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME) para desarrollo local. Exporta el pool para que todos los modelos puedan ejecutar consultas SQL.

config/setup.sql (173 líneas)
Esquema completo de la base de datos con 14 tablas: users (datos del usuario, rol, admin_tag, onboarding), refresh_tokens (tokens de refresco JWT), garments (prendas con color, temporada, imagen, veces usadas), outfits (combinaciones con portada y ocasión), outfit_garments (relación N:M outfits-prendas), calendar_entries (planificación diaria), shared_outfits (publicaciones sociales), likes (likes a publicaciones), comments (comentarios con respuestas anidadas), follows (relaciones de seguimiento con estado pendiente/aceptado), follow_requests (solicitudes), conversations y messages (mensajería directa con soporte multimedia), y support_tickets (tickets de soporte).

Controladores (11 archivos, 1.721 líneas)
Los controladores implementan la lógica de negocio de cada módulo. Reciben las peticiones HTTP, validan los datos, invocan los modelos para interactuar con la base de datos y devuelven las respuestas JSON al cliente.

- **auth.controller.js (205):** Gestiona el registro con hash bcrypt de contraseña, login con generación de JWT (access token + refresh token), refresco automático de tokens y logout con invalidación del refresh token en base de datos.

- **profile.controller.js (262):** El controlador más extenso. Gestiona la visualización y edición de perfiles, subida de avatar con Multer, seguir/dejar de seguir usuarios, aceptar/rechazar solicitudes de seguimiento y la lógica especial de identidad compartida de administradores (cuando un usuario sigue a un admin, el follow se aplica a todos los admins).
- **admin.controller.js (189):** Panel de administración: listar todos los usuarios, crear nuevas cuentas de administrador, inhabilitar/habilitar cuentas de usuario, obtener estadísticas globales de la plataforma.
- **garments.controller.js (176):** CRUD completo de prendas: crear con subida de imagen y detección de color, listar con filtros, actualizar datos, eliminar y marcar como favorito.
- **social.controller.js (176):** Gestión del feed social: obtener publicaciones (modos descubrir/siguiendo), publicar outfits, dar/quitar like, añadir comentarios con respuestas anidadas, eliminar comentarios, buscar publicaciones y usuarios.
- **messages.controller.js (169):** Mensajería directa: crear conversaciones, listar conversaciones con ultimo mensaje y conteo de no leídos, enviar mensajes de texto, subir fotos y audios con Multer, marcar mensajes como leídos.
- **calendar.controller.js (145):** Calendario de planificación: obtener entradas de un mes, asignar outfits o prendas individuales a un día, eliminar entradas. Incrementa automáticamente el contador de "veces usado" de prendas y outfits al asignar el día actual.
- **outfits.controller.js (135):** CRUD de outfits: crear con foto de portada y asociación de prendas, listar con filtros, actualizar datos y prendas asociadas, eliminar cascada.
- **stats.controller.js (111):** Estadísticas del armario: cuenta de prendas por categoría, por temporada, por color, prendas más y menos usadas, outfits más usados.
- **ai.controller.js (103):** Punto de entrada para las funcionalidades de IA: recomendación de outfit basada en clima (envía el armario completo y el clima actual a Gemini), chat conversacional (mantiene historial), y sugerencias de compra (análisis de gaps del armario).

- **support.controller.js (50):** Gestión básica de tickets de soporte: un usuario envía un mensaje y se crea un ticket asociado a su cuenta para revisión por administradores.

Middleware (3 archivos, 89 líneas)

- **auth.middleware.js (32):** Extrae y verifica el token JWT del header Authorization. Si el token es válido, añade el id y rol del usuario al objeto req para uso posterior. Si no es válido o ha expirado, devuelve un 401.
- **admin.middleware.js (18):** Se ejecuta después del auth middleware. Verifica que el usuario autenticado tenga rol "admin". Si no lo tiene, devuelve un 403 Forbidden.
- **upload.middleware.js (39):** Configura Multer para la subida de archivos. Define el directorio de destino (uploads/), genera nombres únicos con timestamp, y establece un filtro para aceptar solo imágenes (jpeg, jpg, png, gif, webp) y archivos de audio (mpeg, wav, webm, mp4, m4a). Límite de 10 MB por archivo.

Modelos (11 archivos, 1.776 líneas)

Los modelos encapsulan todas las consultas SQL a la base de datos. Cada modelo exporta funciones asíncronas que ejecutan queries parametrizadas contra el pool de PostgreSQL.

- **social.model.js (349):** El modelo más extenso. Contiene las queries del feed social: obtener publicaciones con joins a usuarios, likes y comentarios; feed de "siguiendo" filtrado por follows aceptados; crear y eliminar likes; crear comentarios (incluyendo respuestas con parent_id); obtener comentarios con datos del autor; conteo de likes y comentarios; búsqueda de publicaciones por texto.
- **admin.model.js (241):** Queries de administración: listar todos los usuarios con información detallada, contar publicaciones y acciones por administrador, obtener estadísticas globales (total usuarios, posts, likes, comentarios), crear cuentas admin con hash bcrypt, desactivar/activar cuentas, distinguir entre posts eliminados de admins y de usuarios normales.
- **follow.model.js (233):** Queries del sistema de seguimiento: seguir usuario (crear follow con estado pending/accepted según privacidad), dejar de seguir, aceptar/rechazar solicitudes, listar seguidores y siguiendo. Incluye lógica especial para la identidad compartida de administradores:

getFollowersOfAllAdmins y getFollowingOfAllAdmins para agregar los seguidores de todos los admins en un único perfil "Wearo".

- **messages.model.js (189):** Queries de mensajería: crear conversaciones (busca existente o crea nueva), listar conversaciones ordenadas por último mensaje, obtener mensajes de una conversación, enviar mensaje (texto, imagen o audio), marcar como leído, contar mensajes no leídos. El campo medio_type distingue entre text, image y audio.
- **user-profile.model.js (178):** Queries de perfil: obtener datos del usuario por ID, actualizar nombre/bio/avatar, buscar usuarios por nombre o username, completar pasos del onboarding (username, genero, estilos, admin_tag), verificar si un username ya existe. Diferencia entre usuarios normales y admins para el onboarding.
- **outfit.model.js (151):** Queries de outfits: crear outfit con portada, asociar prendas en la tabla pivote outfit_garments, obtener outfits con sus prendas (JOIN), actualizar datos, eliminar con cascada en la tabla pivote, marcar favorito, incrementar veces_usado y last_worn.
- **calendar.model.js (112):** Queries del calendario: obtener entradas de un mes con datos del outfit y prendas, crear entrada (outfit o prendas sueltas), eliminar entrada. Los datos devueltos incluyen el nombre del outfit asignado y las imágenes de sus prendas.
- **stats.model.js (99):** Queries de estadísticas: conteo total de prendas, outfits y favoritos; distribución por categoría con GROUP BY; distribución por temporada; distribución por color; prendas ordenadas por veces_usado (más usadas y menos usadas); outfits más usados.
- **garment.model.js (98):** Queries CRUD básicas de prendas: crear con todos los campos (nombre, categoría, color, marca, temporada, imagen, notas), listar del usuario, obtener por ID, actualizar, eliminar, marcar favorito. La detección de color se hace en el frontend con ColorThief.
- **support.model.js (77):** Queries de soporte: crear ticket con mensaje del usuario, listar todos los tickets (para admin), obtener ticket por ID, marcar como resuelto.

- **user.model.js (49):** Queries básicas de usuario: crear usuario (INSERT con hash bcrypt), buscar por email para login, buscar por ID. Usado exclusivamente por auth.controller.

Servicios (2 archivos, 439 líneas)

- **ai.service.js (329):** Integración completa con Google Gemini. Contiene tres funciones principales: (1) getOutfitRecommendation: construye un prompt detallado con todas las prendas del usuario, el clima actual (temperatura, descripción, ciudad) y las preferencias de estilo, y pide a Gemini que recomiende un outfit especifico del armario; (2) chatWithAI: mantiene un historial de conversación para un chat interactivo sobre moda; (3) getShoppingRecommendations: analiza el armario del usuario e identifica "huecos" (categorías o colores que faltan) para sugerir compras. Todos los prompts se envían en español para obtener respuestas localizadas.
- **weather.service.js (110):** Integración con OpenWeatherMap API. Obtiene el clima actual dado un par de coordenadas (latitud, longitud): temperatura actual, descripción y nombre de la ciudad. También integra Nominatim (OpenStreetMap) para el autocompletado de ciudades: dado un texto parcial, devuelve una lista de ciudades coincidentes con sus coordenadas para que el usuario seleccione la suya.

4.2. Frontend

App.js (15 líneas)

Punto de entrada de la aplicación React Native. Envuelve toda la app con el Provider de Redux (pasando el store global) y el ThemeProvider (para el modo claro/oscuro). Renderiza el AppNavigator como componente raíz.

Pantallas (17 archivos, 9.743 líneas)

Las pantallas son los componentes principales que el usuario ve y con los que interactúa. Cada una corresponde a una ruta del navegador.

- **SocialScreen.js (1.563):** La pantalla más extensa del proyecto. Implementa el feed social completo con dos modos (Descubrir/Siguiendo), renderizado de posts con carrusel de imágenes, sistema de likes con animación, panel de comentarios con respuestas anidadas, compartir posts vía DM, búsqueda de usuarios y publicaciones, modal de detalle de post y lista de usuarios que dieron

like. Cada post muestra avatar, nombre, timestamp, imágenes, contadores y botones de interacción. Incluye la gestión de estados de carga, refresco y estados vacíos.

- **ProfileScreen.js (1.392):** Segunda pantalla más grande. Gestiona tanto el perfil propio como el de otros usuarios. Para el perfil propio: edición, logout, lista de publicaciones, modales de seguidores/siguiendo, botón de solicitudes pendientes con modal de aceptar/rechazar, acceso al panel de admin (si es admin) y acceso a mensajes. Para perfiles ajenos: botón seguir/dejar de seguir (con estado pendiente para cuentas privadas), botón de mensaje directo, contacto con soporte (para admins). Renderiza las publicaciones del usuario en formato feed con likes y comentarios.
- **AdminScreen.js (1.112):** Panel de administración con cuatro pestanas implementadas con estado local. Pestana Usuarios: tabla completa con avatar, nombre, email, fecha, estado, botón inhabilitar y modal para crear nuevos admins. Pestana Contenido: tres subpestañas (publicados, posts usuarios, eliminados) con lista de posts, comentarios expandibles y botones de eliminación. Pestana Soporte: lista de tickets con estado abierto/resuelto. Pestana Stats: métricas globales de la plataforma.
- **ChatScreen.js (992):** Pantalla de chat con mensajería completa. Renderiza mensajes en burbujas (izquierda/derecha según emisor), soporta tres tipos de contenido: texto plano, imágenes (con modal de zoom a pantalla completa) y audio (con controles de reproducción, barra de progreso y duración). Incluye la grabación de audio con temporizador visual, adjuntar fotos desde cámara o galería, visualización de posts compartidos como tarjetas interactivas, y polling automático para recibir mensajes nuevos en tiempo real.
- **CalendarScreen.js (545):** Calendario mensual con navegación mes a mes. Renderiza una cuadricula de días con indicadores de entradas planificadas. Al seleccionar un día muestra las entradas asignadas con carrusel de prendas y botón de eliminar. Modal de asignación con dos pestanas (outfits completos o prendas individuales) para planificar el día actual. Incrementa automáticamente los contadores de uso.
- **HomeScreen.js (509):** Dashboard principal con múltiples secciones: saludo personalizado (buenos días/tardes/noches), tarjeta de clima con temperatura e icono, modal de búsqueda de ciudad con autocompletado, tarjeta de sugerencia

IA, fila de 4 estadísticas rápidas (StatCard), sección de outfit del día con enlace al calendario, carrusel de prendas recientes, y botones de acceso rápido a estadísticas y creación de outfits. Toggle de tema claro/oscuro en la cabecera.

- **AddGarmentScreen.js (488):** Formulario de alta y edición de prendas. Sección de imagen con botones de cámara y galería. Selector de color circular con 18 colores dispuestos en anillo y detección automática mediante ColorThief al seleccionar imagen. Campos de nombre, categoría (con opción de crear personalizada), marca, temporada y notas. Reutilizado tanto para crear como para editar (recibe datos existentes vía params de navegación).
- **OnboardingScreen.js (419):** Flujo de configuración inicial tras registrarse. Para usuarios normales: 3 pasos (username, genero, estilos preferidos). Para administradores: 2 pasos (foto+nombre, admin tag). Navegación con botones anterior/siguiente y puntos de progreso. Validación en cada paso antes de avanzar. Al completar, marca el onboarding como finalizado en la base de datos.
- **OutfitDetailScreen.js (412):** Vista de detalle de un outfit con dos modos: visualización (datos inmutables) y edición (campos editables). En modo visualización muestra portada, ocasión, temporada calculada, veces usado, notas y lista horizontal de prendas. En modo edición permite cambiar nombre, ocasión, notas y seleccionar/deseleccionar prendas en una cuadricula. Botón de eliminar con confirmación.
- **OutfitsScreen.js (331):** Lista de outfits con filtros por ocasión y temporada. Cada tarjeta muestra carrusel de imágenes o portada, nombre, conteo de prendas y ocasión. Toggle de favoritos. Botones de IA: "Chat IA" abre el AIChatModal para asesoramiento, "Que me falta" abre recomendaciones de compra. Botón "+" para crear nuevo outfit.
- **WardrobeScreen.js (315):** Grid de prendas en 2 columnas con filtros por categoría (8 opciones), temporada (5 opciones) y favoritos. Cada tarjeta muestra miniatura, nombre, categoría, marca, fecha de uso y corazón de favorito. Cabecera con conteo de prendas filtradas y botón de anacion. Pull-to-refresh para actualizar la lista.
- **StatsScreen.js (288):** Pantalla de estadísticas con visualización de datos del armario. Muestra tarjetas de resumen (totales), barras de distribución por

categoría con colores diferenciados, distribución por temporada, prendas más usadas y menos usadas en formato lista, y outfits más populares.

- **RegisterScreen.js (280):** Formulario de registro con validación en cliente: nombre obligatorio, email con formato valido, contraseña mínimo 8 caracteres, confirmación de contraseña coincidente. Muestra errores inline. Al registrarse correctamente, redirige al OnboardingScreen.
- **MessagesScreen.js (266):** Lista de conversaciones activas. Buscador superior para encontrar usuarios por nombre (mínimo 2 caracteres, debounce 400ms). Cada conversación muestra avatar, nombre, preview del último mensaje, timestamp y badge de no leídos. Crear nueva conversación al pulsar sobre un resultado de búsqueda. Pull-to-refresh.
- **CreateOutfitScreen.js (256):** Formulario de creación de outfit. Selector de foto de portada opcional. Campo de nombre obligatorio. Tags de ocasión seleccionables. Grid de prendas del armario en 3 columnas con indicador de selección (check badge). Botón de crear habilitado solo con nombre + mínimo 1 prenda seleccionada.
- **LoginScreen.js (247):** Pantalla de inicio de sesión con campos email y contraseña (toggle mostrar/ocultar). Muestra mensajes de error del servidor (credenciales incorrectas) o de red. Botón de login con estado de carga. Enlace a pantalla de registro. Estilos adaptativos al tema.
- **EditProfileScreen.js (181):** Formulario de edición de perfil. Pulsar sobre el avatar abre la galería para cambiar la foto. Campos de nombre y biografía editables. Admin tag editable para administradores. Botón de guardar con estado de carga.
- **GarmentDetailScreen.js (147):** Vista de detalle de una prenda. Imagen grande con overlay del botón atrás. Tarjeta de información con nombre, insignia de favorito, categoría, color, marca, temporada, veces usada y notas. Botones de editar (navega a AddGarmentScreen con datos) y eliminar (con confirmación).

Componentes reutilizables (6 archivos, 1.015 líneas)

- **AIChatModal.js (602):** El componente más grande del frontend. Implementa un chat completo con la IA dentro de un modal. Renderiza mensajes en burbujas (usuario/IA), soporta entrada de texto y grabación de audio (con temporizador y botón grabar/parar). Envía los mensajes al endpoint de IA y muestra las

respuestas. Para las sugerencias de compra, parsea la respuesta y muestra tarjetas de recomendación con categoría, color y motivo. Gestiona el historial de la conversación con scroll automático.

- **GarmentCarousel.js (174):** Carrusel horizontal de imágenes de prendas. Implementado con FlatList para rendimiento optimo. Muestra las fotos de las prendas de un outfit o de una publicación social. Soporta placeholder cuando no hay imagen disponible. Altura configurable. Usado en SocialScreen, OutfitDetailScreen y CalendarScreen.
- **ScreenHeader.js (86):** Cabecera reutilizable para todas las pantallas. Soporta botón de retroceso (con acción personalizable), titulo grande (bold), subtitulo opcional y área de acción derecha (para botones como "+", filtro, etc.). Colores adaptados al tema activo.
- **EmptyState.js (63):** Componente de estado vacío mostrado cuando una lista no tiene datos. Muestra un icono grande (Ionicons), un título, un texto descriptivo y opcionalmente un botón de acción (ej: "Anadir prenda"). Centrado vertical y horizontal. Colores del tema.
- **StatCard.js (58):** Tarjeta compacta para mostrar una métrica. Icono circular (Ionicons) con color personalizable, valor numérico grande y etiqueta descriptiva debajo. Usado en HomeScreen (estadísticas rápidas) y StatsScreen.
- **Card.js (32):** Componente contenedor genérico con bordes redondeados, sombra suave y padding. Soporta onPress para hacerlo pulsable. Color de fondo adaptativo al tema. Base para muchos otros componentes visuales.

Servicios API (11 archivos, 761 líneas)

Los servicios encapsulan todas las llamadas HTTP al backend. Cada servicio utiliza la instancia de Axios configurada en api.js para realizar peticiones autenticadas.

- **api.js (70):** Configura la instancia base de Axios con la baseURL del backend (Railway en producción). Implementa un interceptor de respuesta que detecta errores 401 (token expirado), excluye las rutas /auth/ del proceso automático, y realiza un refresh del token transparente para el usuario. Si el refresh falla, redirige al login.
- **garments.service.js (98):** Funciones para prendas: getAll (con filtros), getById, create (con FormData para imagen), update, delete, toggleFavorite. Create y update utilizan multipart/form-data cuando incluyen imagen.

- **outfits.service.js (88):** Funciones para outfits: getAll, getById, create (con FormData para portada + array de garment_ids), update, delete, toggleFavorite, share (publicar en feed).
- **calendar.service.js (81):** Funciones de calendario: getMonth (entries de un mes), addEntry (con outfit_id o array de garment_ids), removeEntry.
- **social.service.js (78):** Funciones del feed: getFeed (modo discover/following), toggleLike, addComment, deleteComment, getPostComments, searchPosts, searchUsers.
- **messages.service.js (74):** Funciones de mensajería: getConversations, getMessages, sendMessage (texto), sendImage (FormData), sendAudio (FormData), markAsRead, searchUsers.

Los demás servicios (admin, profile, auth, ai, stats) siguen el mismo patrón con funciones específicas para cada módulo.

Store Redux (10 archivos, 1.239 líneas)

El estado global de la aplicación se gestiona con Redux Toolkit. Cada "slice" define el estado inicial, los reducers síncronos y los thunks asíncronos (createAsyncThunk) para las operaciones que requieren llamadas a la API.

- **profileSlice.js (227):** Estado del perfil: datos del usuario, lista de seguidores, lista de siguiendo, solicitudes pendientes. Thunks para fetchProfile, updateProfile, followUser, unfollowUser, acceptRequest, rejectRequest.
- **messagesSlice.js (226):** Estado de mensajería: lista de conversaciones, mensajes de la conversación activa, conteo de no leídos. Thunks para fetchConversations, fetchMessages, sendMessage, sendImage, sendAudio. Incluye lógica de polling que solo actualiza el estado si hay cambios.
- **socialSlice.js (174):** Estado del feed social: array de posts, modo activo (discover/following). Thunks para fetchFeed, toggleLike, addComment, deleteComment. Los reducers actualizan los contadores de likes/comentarios en el array local sin requerir recarga completa.
- **authSlice.js (114):** Estado de autenticación: usuario actual, token de acceso, estado de carga. Thunks para login, register, logout, refreshToken. El logout limpia todo el estado y AsyncStorage.

- **wardrobeSlice.js (116):** Estado del armario: array de prendas, filtros activos (categoría, temporada, favoritos). Thunks para fetchGarments, addGarment, updateGarment, deleteGarment, toggleFavorite.
- **aiSlice.js (127):** Estado de IA: recomendación actual, historial de chat, sugerencias de compra. Thunks para fetchRecommendation, sendChatMessage, fetchShoppingTips.

Los slices de outfits (95), calendar (82), stats (53) e index (25) completan la gestión de estado con sus respectivos thunks y reducers.

Navegación: AppNavigator.js (194 líneas)
Define la estructura de navegación completa de la app con React Navigation. Utiliza un Stack Navigator principal con tres pantallas: LoginScreen, OnboardingScreen y AppTabs. AppTabs es un Tab Navigator inferior con 6 pestanas (Inicio, Armario, Outfits, Calendario, Social, Perfil). Los administradores solo ven Social y Perfil. Cada tab tiene su propio Stack Navigator para manejar la navegación interna (ej: Armario > DetallePreda, Perfil > EditarPerfil > Mensajes > Chat). Iconos de Ionicons con color activo morado (#6C5CE7).

Hook: useTheme.js (140 líneas)
Custom hook que gestiona el tema visual de la aplicación. Define dos paletas de colores completas (claro y oscuro) con valores para: background, surface, text, textSecondary, primary (#6C5CE7), accent (#00CEC9), error (#E17055), warning (#FDCB6E), border y card. Persiste la preferencia del usuario en AsyncStorage. Exporta un ThemeContext que envuelve toda la app y permite a cualquier componente acceder al tema actual y al toggle vía useContext.

5. Resumen de totales

Modulo	Archivos	Líneas
Backend: Configuración	3	283
Backend: Controladores	11	1.721
Backend: Middleware	3	89
Backend: Modelos	11	1.776
Backend: Rutas	11	312
Backend: Servicios	2	439
Frontend: Pantallas	17	9.743
Frontend: Componentes	6	1.015
Frontend: Servicios API	11	761

Frontend: Store Redux	10	1.239
Frontend: Navegación y utilidades	3	349
Archivos de configuración	3	121
TOTAL PROYECTO	91	17.984

El proyecto Wearo consta de un total de 17.984 líneas de código fuente distribuidas en 91 archivos. El frontend representa el 73% del código (13.107 líneas) mientras que el backend supone el 27% (4.620 líneas más 173 de SQL). Las pantallas son el componente más extenso, con 9.743 líneas en 17 archivos, siendo SocialScreen.js (1.563 líneas), ProfileScreen.js (1.392 líneas) y AdminScreen.js (1.112 líneas) los tres archivos más grandes del proyecto.